

IMPLEMENTING AND OPTIMIZING THE SHORTEST VECTOR PROBLEM

FREDDY CUNNINGHAM

1. APPROACH

I chose to use a brute-force approach to solve the shortest vector problem (SVP) as it provides an exact solution and is reasonably simple to implement. The algorithm I implemented was the Schnorr-Euchner Enumeration [1], which uses polynomial-space which is less memory than the other main approach sieving which is an exponential-space algorithm. However, enumeration has a super-exponential run-time [2] but is a natural choice as in practice it typically has good performance on low dimensions.

The run-time of enumeration algorithms is greatly affected by the quality of the input basis. A better basis is shorter and more orthogonal [3]. I, therefore, chose to implement the LLL reduction algorithm [4] to preprocess the lattice basis before running the enumeration algorithm.

I also make use of the Gaussian Heuristic [1] to obtain an upper bound for the length of the shortest vector in the lattice. This is used to define the search space for the enumeration algorithm.

2. IMPLEMENTATION

The primary issue I experienced whilst implementing my solution was working with floating-point values. I chose to implement the classical Gram-Schmidt orthogonalisation (GSO) with floating-point arithmetic (fpa). This is known to be numerically unstable and in the worst case when applied within the LLL algorithm the lattice basis will not necessarily be reduced at all because of inaccuracies in the GSO coefficients [5]. I experienced these problems with my implementation on a number of different types of tests.

In practise a floating-point variant of the LLL algorithm, L^2 is usually implemented which is proven to output a reduced basis in polynomial time. However, I was unable to successfully implement this algorithm.

3. RUN-TIME

To test the run-time of my implementation I generated uniform and knapsack like matrices of dimension 1×1 to 20×20 containing numbers of size 8, 16 and 32 bits using the fplll library.

To measure the run-time of my implementation I used hyperfine a command-line benchmarking tool.

The tests were performed on a virtual machine with 4 cores and 8 GB RAM running an Ubuntu Linux image.

4. MEMORY

To test the amount of memory my implementation used I measured the total bytes used throughout the entire execution of the program as well as the maximum number of bytes used at any point during the programs execution.

This test was repeated for lattices of dimensions 2, 5, 10 and 20 and the results are shown in Table x.

5. ACCURACY

To determine the accuracy of my implementation I compared my solution to the solution generated by the `fpLLL` library. Given the `fpLLL` module uses superior algorithms to deal with floating-point errors, for certain lattices my solution will be less accurate than that produced by `fpLLL`. Therefore, I decided to allow a small tolerance level of 0.00001 that my solution must be within in order to pass.

I ran my implementation on the same tests used to measure run-time and recorded the absolute difference between my solution and that generated by `fpLLL` and whether it passed given my specified tolerance.

REFERENCES

1. Yasuda, M.: A survey of solving SVP algorithms and recent strategies for solving the SVP Challenge. International Symposium on Mathematics, Quantum Theory, and Cryptography. 189 (2020). https://doi.org/10.1007/978-981-15-5191-8_15
2. Gama, P. Q. A. R. O., Nicolas AND Nguyen: Lattice enumeration using extreme pruning. Advances in Cryptology – EUROCRYPT 2010. 257–278 (2010). https://doi.org/10.1007/978-3-642-13190-5_13
3. Kumar, M.: Faster Lattice Enumeration. CoRR. (2019)
4. Plantard, T., Susilo, W., Zhang, Z.: Adaptive precision floating point LLL. Information Security and Privacy. 104 (2013). https://doi.org/10.1007/978-3-642-39059-3_8
5. Nguên, P. Q., Stehlé, D.: Floating-point LLL revisited. Lecture Notes in Computer Science. 215 (2005). https://doi.org/10.1007/11426639_13

DEPARTMENT OF COMPUTER SCIENCE, DURHAM UNIVERSITY
Email address: freddy.t.cunningham@durham.ac.uk